An Analyzable Model for a Visual Novel Engine

Realtime preview, arbitrary jump and robust saves
from a pushdown narrative core

Design proposal · draft for team discussion · 16 October 2026

Summary. Debugging a visual novel (VN) is hard because the correct behaviour lives in the author's head: which sprite should be on screen after a merge, which route a line belongs to, which flags must hold. Existing tools cannot jump to an arbitrary point with correct state. We propose to model a VN as a **deterministic transducer over choice histories**: a pushdown control structure over finite data computes the narrative state, and a pure function maps that state to presentation. This one decision makes saves, preview, jumping and static checking fall out of well-studied theory: pushdown reachability, symbolic model checking with decision diagrams, abstract interpretation and partial-order reduction. The document fixes the model, the restrictions on the script language that keep it analyzable, the save format, the preview mechanism, the linter, and a phased plan. Section 11 lists the questions we need to settle.

Contents

1 Problem and goals 1
2 The core abstraction 2
3 Control model: pushdown control over finite data 3
4 Presentation as a function of state 5
5 Controlling state explosion 6
6 Combinatorial choices: the party problem 7
7 Live preview 8
8 Saving 9
9 Tooling: making intent checkable 9
10 Phased plan 10
11 Open questions for discussion 10
References 11

1 Problem and goals

A VN is mostly linear text with occasional choices, but its state is invisible. A bug report such as "the wrong character is on screen after the roof scene" is only reproducible if one knows *which* path led there, and only diagnosable if one knows what the author *meant* to show. Neither a teammate nor an AI agent has that knowledge today, so debugging falls back to the author replaying the game.

Goal	Requirement
G1 Realtime preview	Editing a line updates the preview at the current position without restarting the game.
G2 Arbitrary jump	Open any line in a *reachable, consistent* state, not a guessed one.
G3 Robust saves	Saves survive script edits, and when they cannot, the engine says exactly where they diverge.
G4 Checkable intent	The author's assumptions become machine-checkable facts that a linter, a teammate or an agent can query.

1.1 Prior art

Two existing tools show both halves of what we want and where each stops.

- **Ren'Py "warp to line"** searches the scene graph for a path to the target line and replays only the `scene`, `show` and `hide` statements on it. Python code on the path is not run, so variables may be uninitialized, and authors patch state by hand in an `after_warp` label. The path search ignores game logic, so the chosen path may not be reachable, and only one path is examined [1].
- **Inky**, the editor for inkle's ink, keeps the player's choices and after each recompile fast-forwards to the last point the author reached [2]. That is trace replay: correct state, but only for the path the author happened to click through.

Our proposal combines the two: Inky-style replay for correctness, plus a logic-aware search (Ren'Py's idea, done soundly) for reaching places the author has not clicked to.

2 The core abstraction

2.1 A VN is a transducer over choice histories

Let Σ be the set of input events, mostly choices, each with a stable identifier. A play session is a word $h \in \Sigma^*$. The engine is defined by three pure pieces:

$$\sigma_h = \mathrm{fold}(\delta, \sigma_0, h) \qquad R = f(\mathrm{pos}, \sigma_h, t)$$

where δ is the narrative transition function, σ is the narrative state (position, call stack, flags), and f maps the current position, state and local time t to a screen R. Everything else in the design follows from keeping δ cheap and total and keeping f free of hidden state.

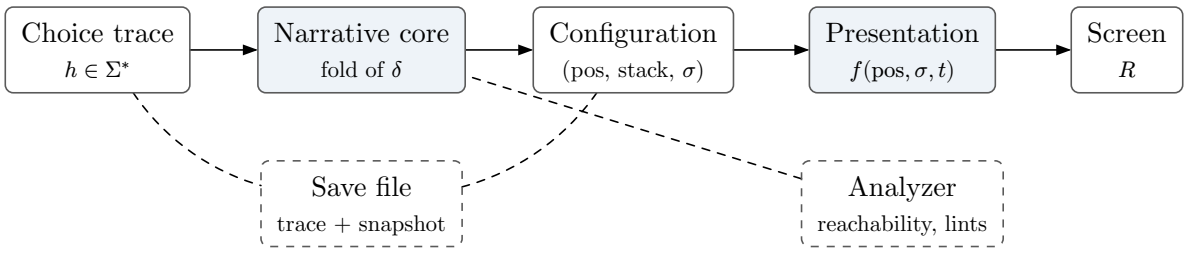


Figure 1: The engine as a pipeline. Only the narrative core is replayed when fast-forwarding; presentation is evaluated lazily at the destination. The analyzer works on the same script IR as the narrative core.

2.2 What should the state be?

There is a principled answer. Two histories h_1, h_2 are **equivalent** when no future input can tell them apart by what appears on screen:

$$h_1 \sim h_2 \iff \forall z \in \Sigma^*.\ \ \mathrm{screens}(h_1 z) = \mathrm{screens}(h_2 z).$$

This is the Myhill–Nerode congruence [3], [4] lifted from acceptors to output-producing machines. The raw history is the largest possible state, the equivalence class $[h]$ is the smallest, and the author's flags sit in between. Most of this proposal is about moving the engine's state toward $[h]$, or avoiding the need to compute it at all.

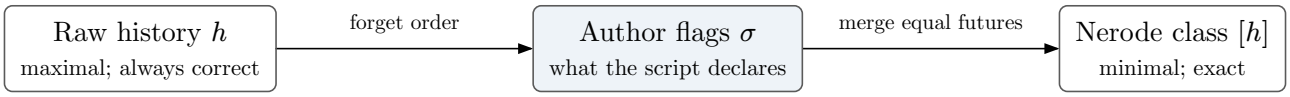


Figure 2: Three granularities of state. Liveness analysis and bisimulation minimization (Section 5.4) move from the middle toward the right.

3 Control model: pushdown control over finite data

3.1 The halting problem comes from data, not from the stack

A pushdown automaton by itself has decidable reachability. Undecidability comes from unbounded *data*: two unbounded counters with conditional jumps already form a Minsky machine, which is Turing-complete without any stack [5]. So the rule is:

- **Control** may be recursive. Scenes call other scenes and return. Loops are allowed (hub maps, day cycles).
- **Data** ranges over finite domains: booleans, enums, bounded or saturating integers.

The result is a pushdown system with finite control, for which reachability (pre* and post*) is computable in polynomial time in the size of the system [6], [7]. One caveat for planning: the size of the system is exponential in the number of boolean variables, which is why we need symbolic representations (Section 5).

What we actually require is weaker than "the game halts", and both properties hold under these rules:

1. **Each step terminates.** Code between two input points is total.
2. **Reachability queries are decidable**, e.g. "can line 412 be reached with `met_B` unset?"

3.2 What a pushdown language looks like

The program-shaped form of a pushdown system is a **Boolean program** (Ball and Rajamani's SLAM project [8], [9]); its graphical twin is a **recursive state machine** (RSM) [10]. In VN terms:

Allowed	**Forbidden (and why)**
Global and scene-local variables of finite type: `bool`, enums, `int` in a declared range	Unbounded integers, runtime strings, lists, maps: they reintroduce counters, hence Turing completeness
`call scene` / `return`, including recursion	Scenes as first-class values, closures: higher-order pushdown systems are decidable but far more expensive to analyze
Computed call over a *finite enum* of scenes (a case split)	Jumping into the middle of another scene: breaks well-nested call/return
`jump` within the current scene; loops that pass through a `choice` or have a fixed bound	Loops with no input and no bound: a single step might not terminate
`choice` (input), `say` / `show` (output via the presentation layer)	Reading wall-clock time or OS state inside the narrative core (see the meta layer, Section 8.3)

Table 1: Restrictions that keep the script language a pushdown system.

A stack frame is a return address plus the caller's locals. Both are finite, so the stack alphabet is finite: that is exactly the definition of a pushdown system.

3.3 When the stack matters: the 学園祭 example

The basic festival hub does **not** need a stack. The player goes from the campus map to stall A, B or C, the story plays, and control always returns to the same place, the map. A revisited stall prints a short message and returns. The only memory is the position and three `visited` bits, so this is a finite automaton.

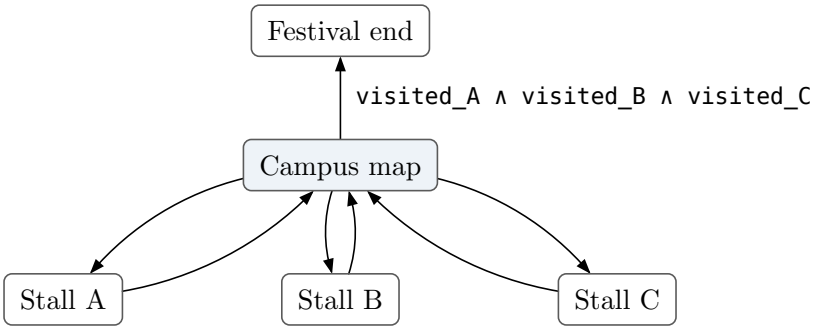


Figure 3: The single-day hub. Every stall returns to the same node, so a plain `jump` suffices. The state space is 5 control nodes × 2^3 flag values.

The stack becomes useful when one sub-scene is entered from different places and must return to different places:

- **Reused hub.** The festival runs on day 1 (afterwards: dorm) and day 2 (afterwards: ending check).
- **Interrupts.** A phone call can happen inside any stall or on the map, and afterwards the game resumes exactly where it was.

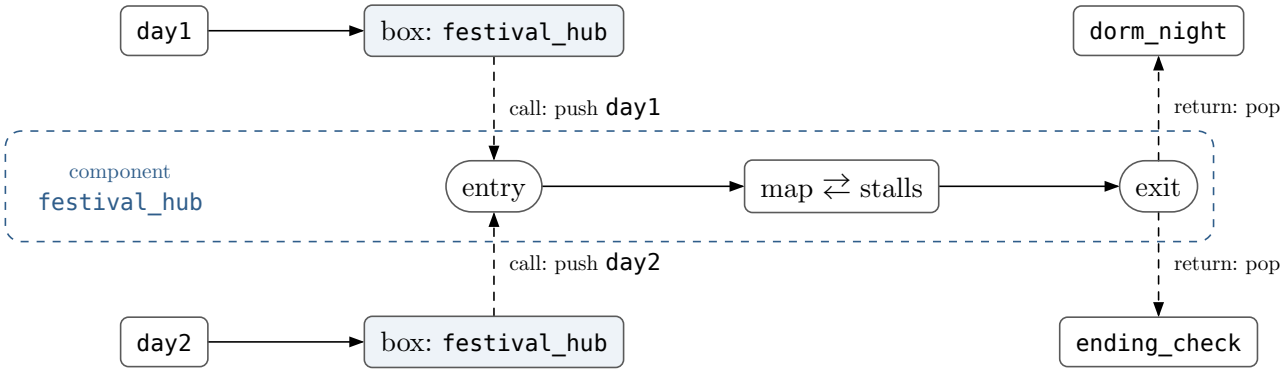


Figure 4: A recursive state machine. Boxes are calls into another component; the stack remembers which box to return to. Without the stack, `festival_hub` would have to be duplicated once per caller.

> **Proposed decision**
> Adopt the pushdown model. Its practical value is **succinctness**: with bounded call depth it recognizes exactly what a finite automaton does, but a finite automaton would need one copy of each shared scene per call site, and with nesting the copies multiply. With unbounded recursion it gains genuine context-free power at no cost to decidability.

```
scene day1:
  ...
  call festival_hub
  jump dorm_night
```

```
scene festival_hub:
  label map:
    choice:
      "Go to A" -> call stall_A ; jump map
      "Go to B" -> call stall_B ; jump map
      "Go to C" -> call stall_C ; jump map
      "Leave"   when visited_A and visited_B and visited_C -> return

scene stall_A:
  if visited_A:
    say "The stall is already packing up."
    return
  visited_A = true
  ...
  return
```

3.4 Node editor as the language

An RSM is literally a node graph: each **component** is a graph with entry and exit nodes, and a **box** stands for a call whose exits are wired to the caller [10]. A component may have several exits; a confession scene can exit through `accepted` or `rejected`, and the caller wires each one differently. The visual editor's graph format *is* the language, and the analyzer works on it directly. A textual form like the one above is a serialization for version control, not a separate language.

4 Presentation as a function of state

4.1 Where the ϕ problem comes from

Imperative commands such as `show bg roof` assign hidden presentation variables. At a point where two branches merge, those variables need a φ-node, exactly as in SSA construction [11]. If the φ is left implicit, the screen depends on the *path*, not on σ, and $f(\text{pos}, \sigma)$ is no longer well defined.

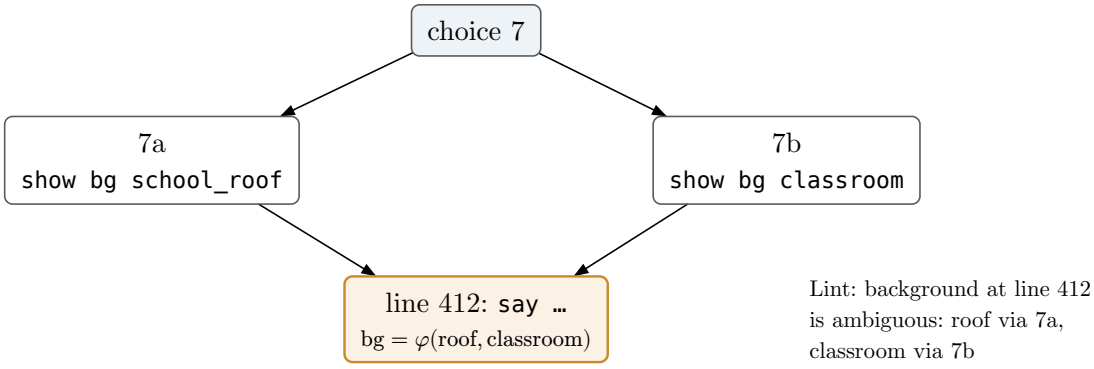


Figure 5: An implicit φ at a merge point. The fix is either a keyframe after the merge or an explicit state variable.

4.2 Three mechanisms, used together

1. **Declarative keyframes.** Any line may declare the complete screen, as in the Elm architecture where the view is a function of the model [12]. Animations are not stored; they are derived by diffing consecutive screens, $\text{transition}(R_{\text{prev}}, R_{\text{next}})$. The previous screen is path-dependent, but it is ephemeral and never saved. The analogy is a video codec: a keyframe is an I-frame, an imperative `show` is a P-frame (a delta), and seeking means jumping to the nearest preceding I-frame and applying the deltas.

2. **Explicit φ promoted into σ.** If the screen after a merge genuinely depends on the branch, that dependency is narrative state and must live in σ, declared by the author or materialized by the engine. Liveness analysis keeps it cheap: the variable dies at the next keyframe.
3. **Ambiguity as a lint.** Abstract interpretation [13] over the script computes, for each line, the set of possible presentation states (powerset lattice, join = union). A singleton is fine; anything larger is reported with the paths that cause it.

> **Proposed decision**
> Presentation is authored in XML (produced by the node editor), as keyframes plus deltas, with conditions that read σ. The narrative core never reads presentation state.

```
<keyframe id="festival.roof.k1">
  <background src="bg/school_roof_evening"/>
  <character id="haruka" pose="smile" at="left"/>
  <switch on="affection.mio">
    <case min="3"><character id="mio" at="right" pose="smile"/></case>
    <default><character id="mio" at="right" pose="neutral"/></default>
  </switch>
  <bgm src="bgm/festival_evening" loop="true"/>
</keyframe>

<delta at="roof.line12">
  <character id="haruka" pose="blush"/>
</delta>
```

The ambiguity lint is likely the single most valuable debugging feature. It catches exactly the class of bug that today "only the author knows about": a sprite left over from another route, BGM that keeps playing after a merge.

5 Controlling state explosion

Twenty independent three-way choices give $3^{20} \approx 3.5 \times 10^9$ states; we estimate that commercial titles such as 君と彼女と彼女の恋。 [14] have branching on this scale. That number only matters if we enumerate states, and we never will.

5.1 Saves and jumps are traces, not states

To show line X we do not need "the state at X"; we need *some* history that reaches X, then a replay. Replaying δ without presentation is cheap (no rendering, no animation, no asset loading), and f is evaluated once, at the destination.

5.2 Sets of states are symbolic

The set of states that can reach a line is represented as a reduced ordered binary decision diagram [15], or a multi-valued decision diagram for enum flags [16]. Twenty independent three-way choices give a diagram with about twenty nodes. Blowup only happens when flags interact in genuinely complicated ways, and those interactions are what the author needs to see anyway. Each line in the editor can display its **reachability condition**, for example $(\text{met}_A \wedge \neg\text{met}_B) \vee \text{route} = C$.

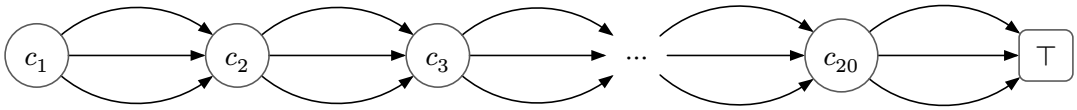


Figure 6: A multi-valued decision diagram for twenty independent three-way choices: 21 nodes and 60 edges, against 3^{20} leaves for an explicit tree.

5.3 Jumping is witness generation

"Show me line X in the case where Mio's affection is at least 3" is a model-checking query [17]. The decision diagram (or a SAT solver) returns a satisfying assignment and a path that realizes it, which is then replayed. An unreachable line is dead code and is reported.

5.4 Shrinking the state with dataflow analysis

- **Liveness.** A flag is dead after its last read, so it is projected out of reachable-set computations from that point. Interprocedural versions of these analyses are graph reachability problems [18].
- **Bisimulation minimization.** States with identical futures are merged by partition refinement [19]. This is the computable approximation of the Nerode quotient. In practice most early flags collapse after a route lock.

5.5 Lazy, incremental evaluation

Nothing above is computed up front. Each analysis is a memoized, demand-driven query such as "reachable set at node N" or "presentation at N under σ", in the style of self-adjusting computation [20] and Adapton [21], for which Salsa [22] is a production implementation. Editing one scene invalidates only the queries that depend on it, which is what makes realtime preview affordable while editing.

6 Combinatorial choices: the party problem

At a party the player talks to each of n guests once, in any order. There are $n!$ complete orderings and $\lfloor e \cdot n! \rfloor$ ordered prefixes. If the future depends only on *who* has been visited, the Nerode state is the visited subset: 2^n states, the same compression the Held–Karp algorithm uses for the travelling salesman problem [23].

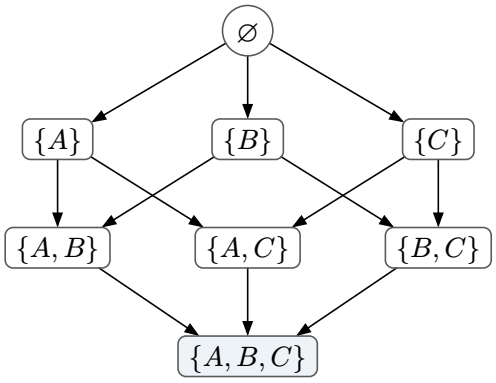


Figure 7: Subset lattice for $n = 3$. Each path is one ordering; states are shared.

n	**ordered prefixes**	**subsets 2^n**	**counts $n+1$**
3	16	8	4
5	326	32	6
8	109 601	256	9
10	9 864 101	1 024	11

Table 2: States needed under three assumptions about what the future depends on.

Further reductions, in order of strength:

- **Independence.** If conversations A and B touch disjoint parts of σ, then AB and BA reach the same state. This is the theory of Mazurkiewicz traces [24], [25], and partial-order reduction explores one representative per equivalence class [26], [27], [28].
- **Symmetry.** If only the *number* of visited guests matters, there are $n + 1$ states.
- **Symbolically** the set "all subsets of n independent bits" is the constant-true diagram.

6.1 A shuffle operator in the language

The important move is in the language, not the analyzer. A finite or pushdown automaton can express "each guest exactly once" only by spelling out 2^n control states. A **shuffle** (interleaving) operator stays in the same language class (the shuffle of regular languages is regular) but is exponentially more succinct [29]:

```
scene party:
  each_once until leave_party:
    talk(haruka)
    talk(mio)
    talk(rin)
    talk(sensei)
```

The shuffle of context-free languages is not context-free in general [30], so each branch must run **atomically**: a conversation is a call that returns before the next one starts. Order-dependence is then declared rather than discovered:

```
scene talk_mio:
  if visited(haruka):        # declared dependency on the visited set
    say "Haruka told me you'd come."
  ...
```

The analyzer reads the state as a bitset plus the declared dependencies. It never explores orderings to find out that they commute.

7 Live preview

> **Proposed decision**
> Snapshots are taken at choice points. Preview re-executes the narrative core from the nearest preceding choice snapshot to the cursor, then evaluates f there. Locating *which* line to inspect is left to the author, supported by the linter; witness-based jumping (Section 5.3) is a later phase.

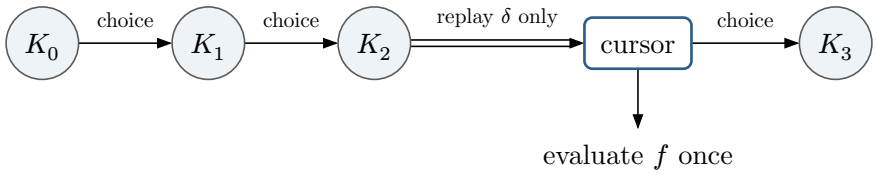


Figure 8: Preview after an edit: restore snapshot K_2, replay the narrative core to the cursor, render once.

Two details:

- **The snapshot includes the call stack.** After the choice, a `return` must know where to go. Inside the hub that is one frame; inside a phone call during stall A on day 2 it is three.
- **Hot reload.** When the author edits a node, snapshots after the first edited node on the current trace are invalidated and rebuilt by replay, as Inky does after recompiling [2]. If a recorded choice no longer exists, preview stops at that choice and highlights it.

8 Saving

8.1 Format

Part	Content	Role
Trace	Sequence of stable choice IDs, plus script version hash and PRNG seed	canonical
Snapshot	Position, call stack, σ, promoted φ variables, PRNG state	cache

Choice IDs are stable identifiers assigned when a choice node is created, never line numbers. Randomness uses a deterministic PRNG whose state is part of σ, so replay is exact.

8.2 Loading across script versions

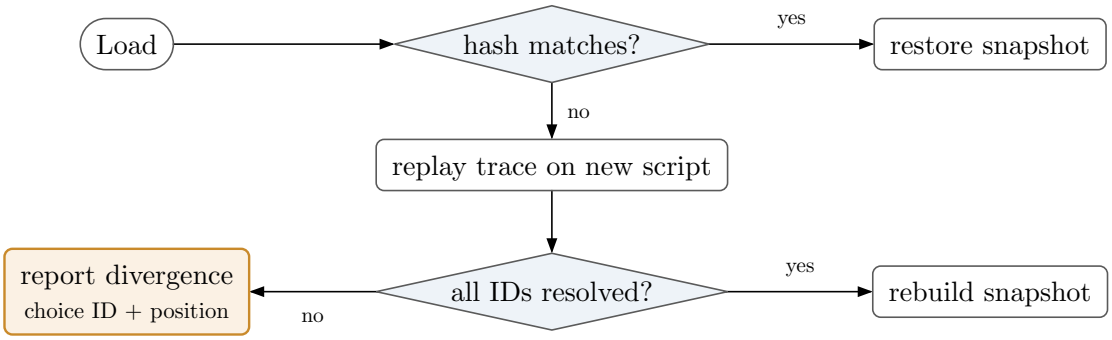


Figure 9: Loading a save. The snapshot is only trusted when the script is unchanged.

8.3 The meta layer

Persistent cross-playthrough state (unlocks, read-text flags, gallery) lives in a separate outer layer: a parameter the run reads at start and writes at defined points. Some titles deliberately make this layer part of the story; 君と彼女と彼女の恋。 is a well-known example [14]. For such games the meta layer is part of the semantics, so the analyzer treats its fields as ordinary finite variables and the debugger lets the author set them directly.

9 Tooling: making intent checkable

9.1 Lints that need no annotations

Lint	Catches	Technique
Unreachable node	Dead lines, impossible route conditions	pushdown reachability
Ambiguous presentation	Leftover sprite/BGM/background after a merge	abstract interpretation
Write-never-read	Flags that never influence anything	liveness
Read-before-write	Flags read on a path where no write happened	reaching definitions
Unbounded step	Loop without input or bound	syntactic check
Non-atomic shuffle	`each_once` branch that can be interrupted mid-way	call-structure check

9.2 Contracts

Authors can attach assertions to nodes: “Haruka is on screen here”, “this scene requires `met_B`”, “after the confession, `route` never changes again”. These are temporal-logic properties [31], checked over all

reachable states. A failing contract comes with a counterexample trace, which opens directly in the previewer.

9.3 Why this helps AI agents

An agent fails today because author intent is not written down in a checkable form. With this model an agent can query reachability conditions, read contracts, run lints, replay counterexamples, and propose missing contracts ("line 412 is reached with two different backgrounds; should it have a keyframe?"), instead of guessing.

10 Phased plan

Each phase is usable on its own and does not require the later ones.

Phase	Deliverable	Unlocks
0	Script IR as RSM (node graph + text serialization); narrative interpreter; trace saves with snapshot cache	G3
1	XML presentation (keyframes, deltas, `when` conditions); diff-based transitions; choice-point snapshots and hot-reload preview	G1
2	Explicit-state analyzer for small scripts; unreachable, ambiguity, liveness lints	G4 (partial)
3	Symbolic reachability (MDDs); per-line reachability conditions; witness jump; contracts with counterexamples	G2, G4
4	Incremental query engine for live analysis; `each_once` with declared dependencies; partial-order reduction	scale

Phase 2 deliberately starts with explicit-state search: it is simple, it validates the IR, and for small projects it is fast enough. Phase 3 replaces the backend without changing the lint interface.

11 Open questions for discussion

Open question

1. **Integer ranges.** Are bounded integers saturating or is overflow a lint error? What default range do affection counters get?
2. **Recursion depth.** Allow unbounded recursion (full pushdown) or cap stack depth (finite automaton, simpler analysis)? No current use case needs unbounded recursion.
3. **Timed input.** Are auto-advance, timed choices and QTEs events in Σ? If so, how are they recorded in traces?
4. **φ policy.** When the ambiguity lint fires, should the editor offer to insert a keyframe, to promote a state variable, or both?
5. **Source of truth.** Is the node graph or the text serialization canonical in version control? Graph merges are hard; text diffs are easy to review.
6. **Choice IDs.** Generated identifiers or author-chosen names? Generated IDs are stable under renaming; names are readable in divergence reports.
7. **Meta layer scope.** Which meta fields may the narrative core read, and at which points may it write them?

References

[1] Ren'Py contributors, "Ren'Py Documentation: Developer Tools (Warping to a Line)." [Online]. Available: https://github.com/renpy/renpy/blob/master/sphinx/source/developer_tools.rst

[2] inkle, "Inky: An Editor for ink, inkle's Markup Language for Writing Interactive Narrative." [Online]. Available: https://github.com/inkle/inky

[3] A. Nerode, "Linear Automaton Transformations," *Proceedings of the American Mathematical Society*, vol. 9, no. 4, pp. 541–544, 1958.

[4] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Addison-Wesley, 2006.

[5] M. L. Minsky, *Computation: Finite and Infinite Machines*. Prentice-Hall, 1967.

[6] A. Bouajjani, J. Esparza, and O. Maler, "Reachability Analysis of Pushdown Automata: Application to Model-Checking," in *CONCUR '97: Concurrency Theory*, in LNCS, vol. 1243. Springer, 1997, pp. 135–150.

[7] S. Schwoon, "Model-Checking Pushdown Systems," Doctoral dissertation, 2002.

[8] T. Ball and S. K. Rajamani, "Boolean Programs: A Model and Process for Software Analysis," technical report MSR-TR-2000-14, 2000.

[9] T. Ball and S. K. Rajamani, "Bebop: A Symbolic Model Checker for Boolean Programs," in *SPIN Model Checking and Software Verification*, in LNCS, vol. 1885. Springer, 2000, pp. 113–130.

[10] R. Alur, M. Benedikt, K. Etessami, P. Godefroid, T. Reps, and M. Yannakakis, "Analysis of Recursive State Machines," *ACM Transactions on Programming Languages and Systems*, vol. 27, no. 4, pp. 786–818, 2005.

[11] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently Computing Static Single Assignment Form and the Control Dependence Graph," *ACM Transactions on Programming Languages and Systems*, vol. 13, no. 4, pp. 451–490, 1991.

[12] E. Czaplicki and S. Chong, "Asynchronous Functional Reactive Programming for GUIs," in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2013, pp. 411–422.

[13] P. Cousot and R. Cousot, "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints," in *Proceedings of the 4th ACM Symposium on Principles of Programming Languages (POPL)*, 1977, pp. 238–252.

[14] Nitroplus, "君と彼女と彼女の恋。 (Totono)." 2013.

[15] R. E. Bryant, "Graph-Based Algorithms for Boolean Function Manipulation," *IEEE Transactions on Computers*, vol. C-35, no. 8, pp. 677–691, 1986.

[16] A. Srinivasan, T. Kam, S. Malik, and R. K. Brayton, "Algorithms for Discrete Function Manipulation," in *IEEE International Conference on Computer-Aided Design (ICCAD)*, 1990, pp. 92–95.

[17] C. Baier and J.-P. Katoen, *Principles of Model Checking*. MIT Press, 2008.

[18] T. Reps, S. Horwitz, and M. Sagiv, "Precise Interprocedural Dataflow Analysis via Graph Reachability," in *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1995, pp. 49–61.

[19] R. Paige and R. E. Tarjan, "Three Partition Refinement Algorithms," *SIAM Journal on Computing*, vol. 16, no. 6, pp. 973–989, 1987.

[20] U. A. Acar, “Self-Adjusting Computation,” Doctoral dissertation, 2005.

[21] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster, “Adapton: Composable, Demand-Driven Incremental Computation,” in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014, pp. 156–166.

[22] Salsa contributors, “Salsa: A Generic Framework for On-Demand, Incrementalized Computation.” [Online]. Available: https://github.com/salsa-rs/salsa

[23] M. Held and R. M. Karp, “A Dynamic Programming Approach to Sequencing Problems,” *Journal of the Society for Industrial and Applied Mathematics*, vol. 10, no. 1, pp. 196–210, 1962.

[24] A. Mazurkiewicz, “Concurrent Program Schemes and Their Interpretations,” DAIMI Report PB-78, 1977.

[25] V. Diekert and G. Rozenberg, Eds., *The Book of Traces*. World Scientific, 1995.

[26] A. Valmari, “Stubborn Sets for Reduced State Space Generation,” in *Advances in Petri Nets 1990*, in LNCS, vol. 483. Springer, 1991, pp. 491–515.

[27] D. Peled, “All from One, One for All: On Model Checking Using Representatives,” in *Computer Aided Verification (CAV '93)*, in LNCS, vol. 697. Springer, 1993, pp. 409–423.

[28] P. Godefroid, *Partial-Order Methods for the Verification of Concurrent Systems: An Approach to the State-Explosion Problem*, vol. 1032. in LNCS, vol. 1032. Springer, 1996.

[29] A. J. Mayer and L. J. Stockmeyer, “The Complexity of Word Problems — This Time with Interleaving,” *Information and Computation*, vol. 115, no. 2, pp. 293–311, 1994.

[30] J. Gischer, “Shuffle Languages, Petri Nets, and Context-Sensitive Grammars,” *Communications of the ACM*, vol. 24, no. 9, pp. 597–605, 1981.

[31] A. Pnueli, “The Temporal Logic of Programs,” in *18th Annual Symposium on Foundations of Computer Science (FOCS)*, 1977, pp. 46–57.